

笔记结合仓库 `sia-rgb/miniGPT-training-lab` 里的 **nanoGPT 训练代码**来复盘。这个项目本质上是在 Tiny Shakespeare 数据集上训练一个字符级 GPT：把文本拆成字符，每个字符映射成整数，然后让模型不断练习“根据前面的字符预测下一个字符”。

1. 整体训练流程：GPT 训练到底在循环什么？

nanoGPT 的训练循环可以理解成一句话：

每次从训练数据里随机抽一批连续文本片段，让模型预测每个位置的下一个字符；预测错了就算 loss，再用 backward 算出每个参数该怎么改，最后由 optimizer 更新参数。循环很多次以后，模型就逐渐学会了莎士比亚文本的统计规律。

对应代码主线是：

```
X, Y = get_batch('train') ## 从train.bin里取出一批训练样本

while True: ## 开始一个无限训练循环
    lr = get_lr(iter_num) ## 根据当前训练到第几步，计算这一轮要用的学习率。

    ``if iter_num % eval_interval == 0:`` ## 每隔固定步数(被整除)，做一次评估
        ``losses = estimate_loss()`` ## 暂停一下正式训练，评估模型当前水平
        ``maybe_save_checkpoint()`` ## 如果当前模型表现不错，就保存一次训练存档

    ``logits, loss = model(X, Y)`` ## 把 `X` 喂给模型，让模型预测 `Y`，并计算预测错了多少
    ``X, Y = get_batch('train')`` ## 取新的一批训练数据
    ``loss.backward()`` ## 根据 loss 进行反向传播，计算每个参数的梯度
    ``optimizer.step()`` ## 优化器根据刚刚算出来的梯度，真正更新模型参数
    ``optimizer.zero_grad()`` ## 清空上一轮计算出来的梯度
    ``iter_num += 1`` ## 当前训练轮数加 1
```

这段训练循环的本质是：

X 是题目，Y 是答案；
model(X, Y) 是做题并算错多少；
loss.backward() 是分析错因；
optimizer.step() 是根据错因修改模型参数；
optimizer.zero_grad() 是清空本轮错因分析；
iter_num += 1 是进入下一轮训练。

也可以压缩成 nanoGPT 的训练闭环：

batch

```
→ forward
→ loss
→ backward
→ optimizer update
→ zero grad
→ eval/checkpoint
→ next batch
```

真实代码里还包含混合精度、梯度累积、梯度裁剪、DDP 分布式训练等工程细节，**但核心逻辑就是上面这条链路。训练循环从 `get_batch('train')` 先取第一批数据开始，然后进入 `while True` 主循环；每轮会设置学习率、定期评估 loss 和保存 checkpoint，再执行 forward、backward、optimizer step。

2. 数据如何变成 batch(一批训练样本)?

项目使用的是 `data/shakespeare_char/prepare.py`。这一步不是训练本身，而是训练前的数据准备。

原始文本先被读入，然后取出所有出现过的字符，建立两个映射：

```
stoi = { ch:i for i,ch in enumerate(chars) }
itos = { i:ch for i,ch in enumerate(chars) }
```

`stoi` 是“string to integer”，把字符变成数字；`itos` 是“integer to string”，把数字还原成字符。比如字符 "A" 可能被映射成某个整数。代码里还把数据按 90% / 10% 分成训练集和验证集，分别保存成 `train.bin` 和 `val.bin`，并把词表信息保存到 `meta.pkl`。

在训练时，`train.py` 里的 `get_batch(split)` 会读取这些 `.bin` 文件：

```
if split == 'train':
    data = np.memmap(... 'train.bin' ...)
else:
    data = np.memmap(... 'val.bin' ...)
```

然后随机抽取 `batch_size` 个起点：

```
ix = torch.randint(len(data) - block_size, (batch_size,))
```

接着，对每个起点 `i`，截取长度为 `block_size` 的序列作为 `x`，再把同一段序列整体向后平移 1 位作为 `y`：

```
x = data[i : i + block_size]
y = data[i + 1 : i + 1 + block_size]
```

代码中真实写法是用 `torch.stack` 把多个样本堆成 batch。

结合配置文件，`batch_size = 64`，`block_size = 256`，所以每个 batch 的形状大致是：

```
x.shape = [64, 256]
y.shape = [64, 256]
```

也就是说，一次训练不是只看一个字符，而是同时看 64 条训练样本，每条样本长度 256 个字符。配置文件中还明确设置了 `dataset='shakespeare_char'`、`gradient_accumulation_steps=1`、`batch_size=64`、`block_size=256`。

3. x 和 y 是什么？

`x` 是模型看到的输入，`y` 是模型要预测的标准答案。

举一个极简例子。假设原始文本是：

```
hello
```

如果 `block_size = 4`，那么可能取到：

```
x = h e l l
y = e l l o
```

也就是：

位置	模型看到的 x	正确答案 y
第 1 位	h	e
第 2 位	e	l
第 3 位	l	l
第 4 位	l	o

这就是 GPT 的核心训练任务：**next-token prediction**，即“下一个 token 预测”。

项目里是字符级模型，所以 token 就是字符；如果是 GPT-2 / GPT-3 这类模型，token 通常是 BPE 子词片段，不一定是单个字符。

所以，`x` 和 `y` 的关系不是两份不同文本，而是同一段文本错开一位：

```
x: To be, or not to by: o be, or not to be
```

模型看到 `x`，目标是让每个位置都预测出 `y` 对应位置的字符。

4. forward 在预测什么？

`forward` 的输入是 `idx`，也就是 `batch` 里的 `x`。模型会先把整数 `token` 变成向量，再加上位置向量：

```
tok_emb = self.transformer.wte(idx) ## 把每个字符编号，转换成一个模型能理解的向量
pos_emb = self.transformer.wpe(pos) ## 给每个字符加上它在句子里的位置信息
x = self.transformer.drop(tok_emb + pos_emb) ## 告诉模型“哪个字符在什么位置”
```

然后经过多层 `Transformer Block`：

```
for block in self.transformer.h: ## 把刚才的字符向量，送进一层又一层Transformer里加工
    x = block(x)
```

最后通过 `lm_head` 映射到词表大小：

```
logits = self.lm_head(x) ## 把模型内部加工后的向量，转换成对词表中每个字符的打分
```

在你的 `Shakespeare` 字符级任务里，词表大小是 65，也就是一共有 65 种可能字符。

`prepare.py` 的注释输出显示 `vocab size` 为 65。

所以 `logits` 的含义是：

对 `batch` 中每一条样本、每一个位置，模型都输出一个长度为 `vocab_size` 的分数向量。

如果形状展开来看：

```
logits.shape = [batch_size, block_size, vocab_size] = [64, 256, 65]
```

通俗理解：

模型不是直接吐出一个字符，而是对 65 个候选字符分别打分。哪个字符分数更高，模型就更认为它可能是下一个字符。

在训练阶段，`model(X, Y)` 会同时返回 `logits` 和 `loss`；在推理生成阶段，如果没有传入 `targets`，代码只计算最后一个位置的 `logits`，用于预测下一个 token。

5. loss 是怎么算的？

训练时，模型有预测结果 `logits`，也有标准答案 `targets`，也就是 `Y`。于是可以比较：

```
loss = F.cross_entropy(
    logits.view(-1, logits.size(-1)),
    targets.view(-1),
    ignore_index=-1
)
```

这是交叉熵损失。

通俗理解：

如果正确答案是字符 "e"，模型给 "e" 的概率越高，loss 越低；如果模型把概率给了别的字符，loss 就越高。

比如某个位置的正确答案是 "o"：

```
模型预测：
h: 0.05
e: 0.10
l: 0.20
o: 0.60
其他: ...
```

这时 loss 比较低，因为正确答案 "o" 的概率高。

如果模型预测成：

```
h: 0.60
e: 0.10
l: 0.20
o: 0.01
其他: ...
```

这时 loss 很高，因为正确答案 "o" 的概率只有 0.01。

注意，loss 不是简单的“错了几个字符”。它更细：它衡量的是模型给正确答案分配了多少概率。

6. backward 在干什么？

backward 对应代码是：

```
scaler.scale(loss).backward()
```

如果不考虑混合精度，可以理解成：

```
loss.backward()
```

它做的事情是：**反向传播，计算梯度**。

更通俗地说：

loss 告诉我们“现在错得有多离谱”，backward 告诉我们“每个参数分别对这个错误负了多少责任，以及应该往哪个方向改”。

模型里有很多参数，比如 embedding 权重、attention 权重、MLP 权重、LayerNorm 参数等。forward 之后，loss 是一个数字；backward 会沿着计算图反向追踪，算出：

如果某个参数稍微变大一点，loss 会变大还是变小？
如果变小，应该变大多少？
如果变大，应该反方向调整多少？

这个“参数应该怎么变”的方向和强度，就是梯度。

所以，backward 本身并不更新参数，它只是把每个参数的 `.grad` 算出来。

7. optimizer 为什么更新参数？

backward 只是算出“该怎么改”，optimizer 才是真的动手改模型参数。

训练一个 GPT，本质上就是不断调整模型里面的大量参数，让它下一次预测得更准、loss 更低。

模型参数是什么？

可以先把模型想象成一个巨大的“预测机器”。

这个机器里面有很多可以调节的旋钮：

```
embedding 参数
attention 参数
MLP 参数
LayerNorm 参数
lm_head 参数...
```

这些旋钮就是模型参数。

刚开始训练时，这些参数基本是随机初始化的。

所以模型一开始预测字符时，基本是在乱猜。

真正改参数的是：

```
scaler.step(optimizer) ## 根据错因，修改自己知识结构
scaler.update() ## 如果使用混合精度训练，就更新数值缩放策略，保证训练稳定
optimizer.zero_grad(set_to_none=True) ## 把这次错题分析清空，准备下一轮训练
```

不考虑混合精度时，可以理解成：

```
optimizer.step() ## 根据刚刚算出来的梯度，真正修改模型参数
optimizer.zero_grad() ## 清空本轮梯度，准备下一轮重新计算
```

模型通过：

```
optimizer = model.configure_optimizers(...) ## 接下来训练时，用哪种“改参数的方法”
```

创建优化器，`model.py` 里用的是 `torch.optim.AdamW`。代码还把参数分成 `weight decay` 和 `no weight decay` 两组：二维以上的权重矩阵会做 `weight decay`，`bias` 和 `LayerNorm` 这类一维参数不做。

`AdamW` 是一种比较常用、比较稳的参数更新方法。

最简单的优化器可能只会看：

这一轮梯度指向哪里，我就往反方向走一步。

但是 `AdamW` 更聪明一点。

它不只看当前这一轮，还会参考历史梯度：

```
`最近几轮参数是不是都该往同一个方向改？
这个参数是不是变化太剧烈？
有些权重是不是变得太大，需要压一压？`
```

所以 AdamW 像一个有经验的教练：

它不是看一次错题就大幅调整，而是结合最近多次表现，比较稳地调整参数。

可以先不记 AdamW 的数学公式，只记直觉：

AdamW = 更稳定、更常用的参数更新方法。

weight decay 可以理解成：

给参数加一个“别长太大”的约束。

如果模型的某些权重变得特别大，模型可能会过度自信，也更容易过拟合。

比如它可能在训练集里记住了某些特殊模式，但泛化能力不好。

weight decay 的作用是：

参数可以变，但不要无限变大；模型可以学习，但不要太极端。

通俗类比：

学生可以根据错题调整理解，但不能因为一道题就把某个规则过度放大。

通俗理解：

backward 算出“该往哪里改”，optimizer 负责“真的改一下”。

参数更新的大概形式可以理解成：

新参数 = 旧参数 - 学习率 × 梯度方向

‘旧参数：模型现在的状态’

‘梯度方向：loss 上升最快的方向’

‘学习率：这一步迈多大’

所以训练的本质是循环：

预测 → 计算错误 → 找到责任参数 → 调整参数 → 再预测 → 再调整

重复很多次以后，模型的参数就从“随机初始化”变成“能预测莎士比亚风格文本的参数”。

8. eval loss 和 train loss 的区别

在 `train.py` 里，`estimate_loss()` 会分别在 `train` 和 `val` 上计算 loss：

```
for split in ['train', 'val']:
    X, Y = get_batch(split)
    logits, loss = model(X, Y)
```

它会先 `model.eval()`，算完后再 `model.train()`。

这里有两个关键点。

第一，**train loss 是在训练集上算的 loss**。训练集是模型不断见过、不断用来更新参数的数据。因此 train loss 下降，说明模型越来越会拟合训练数据。

第二，**eval / val loss 是在验证集上算的 loss**。验证集没有参与参数更新，相当于“新题”。所以 val loss 更能反映模型是否真的学到了**泛化规律**，而不是只背下训练文本。

可以这样理解：

```
train loss: 课堂练习题上的表现
val loss: 没见过的模拟考试题上的表现
```

如果出现：

```
train loss 持续下降
val loss 先下降后上升
```

这通常说明模型开始过拟合：它会越来越会背训练集，但对新文本的预测能力变差。

你的配置里 `eval_interval = 250`，也就是每 250 次迭代评估一次；`eval_iters = 200`，表示评估时会抽 200 个 batch 求平均 loss，让评估结果更稳定。

9. checkpoint 保存什么？

checkpoint 是训练过程中的“存档点”。在 `train.py` 中，当验证集 loss 创新低，或者设置了 `always_save_checkpoint=True` 时，会保存：

```
checkpoint = {
    'model': raw_model.state_dict(),
    'optimizer': optimizer.state_dict(),
    'model_args': model_args,
    'iter_num': iter_num,
```

```
    'best_val_loss': best_val_loss,
    'config': config,
}
torch.save(checkpoint, os.path.join(out_dir, 'ckpt.pt'))
```

也就是说，checkpoint 至少保存这些东西：

保存内容	含义	为什么重要
model	模型参数	这是训练出来的核心成果
optimizer	优化器状态	用于恢复 AdamW 的动量等状态
model_args	模型结构参数	恢复时要知道层数、头数、hidden size、block size 等
iter_num	当前训练到第几步	用于断点续训
best_val_loss	历史最好验证 loss	用于判断后续是否有改进
config	本次训练配置	方便复盘和复现实验

通俗理解：

checkpoint 不只是“模型权重文件”，而是一次训练现场的完整快照。
有了它，既可以拿来生成文本，也可以从中断处继续训练。

这也是你部署和训练项目里很重要的工程意识：如果训练 8 小时，中途断了但没有 checkpoint，就只能重来；如果保存了 checkpoint，就可以 resume。

10. 把一次训练 iteration 串起来看

现在把所有概念串成一条完整链路：

1. 从 `train.bin` 随机抽 64 个起点
2. 每个起点截取 256 个字符，形成 `X`
3. 把同一段文本向后错一位，形成 `Y`
4. `X` 输入 GPT
5. GPT 对每个位置预测“下一个字符”的概率分布
6. `logits` 和 `Y` 做 `cross entropy`，得到 `loss`
7. `backward` 根据 `loss` 计算所有参数的梯度
8. `optimizer` 根据梯度更新参数
9. 清空梯度，进入下一轮
10. 每隔 `eval_interval`，在 `train/val` 上评估 `loss`
11. 如果 `val loss` 更好，就保存 `checkpoint`

可以压缩成一句更工程化的话：

```
Input: X batch
Target: Y shifted batch
Process: forward → loss → backward → optimizer.step
Output: updated model parameters + periodic checkpoint
```

11. 最重要的直觉：GPT 不是“被教会回答”，而是“被训练成预测下一个 token”

在这个 miniGPT 训练里，模型没有被教“什么是莎士比亚”，也没有被人工写规则告诉它“角色名后面应该跟台词”。

它只是在做一个非常朴素的事情：

给定前面的字符，预测下一个字符。

但是当这个任务被重复几千次、几百万次、几百亿次后，模型为了更好地预测下一个 token，就被迫学会：

单词拼写
标点习惯
句子结构
角色名格式
对话换行
莎士比亚文本风格
局部语义关系

所以 GPT 训练的神奇之处在于：

训练目标很简单：预测下一个 token。

但为了完成这个目标，模型内部会逐渐形成对语言结构的压缩表示。

nanoGPT 项目正好是这个机制的最小可运行版本。它没有大模型那么强，但训练闭环是同一套：

batch → forward → loss → backward → optimizer → eval → checkpoint。